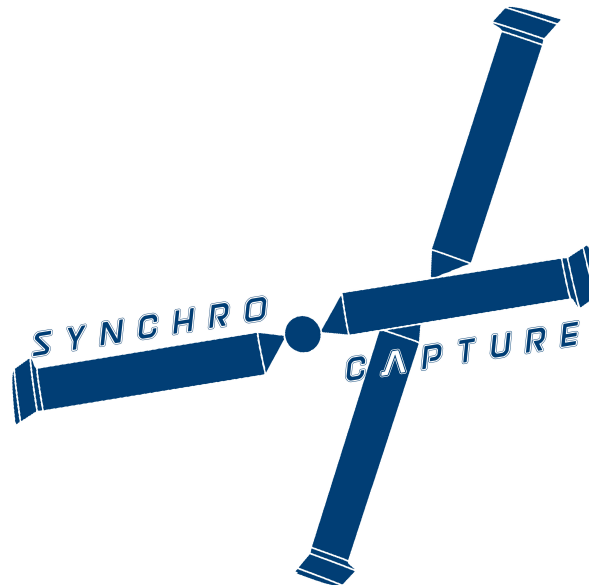


AERO96005 - GROUP DESIGN PROJECT
WHOLLY REUSABLE LAUNCH SYSTEM (RECOVERY)

**FLIGHT SIMULATION OF AERIAL
CAPTURE AND HANDLING QUALITY
FOR A SYNCHROPTER CONCEPT FOR
THE AERIAL-RECOVERY OF FALLING
ROCKET STAGES**



Student: Hakan Serpen
CID: 01335573

Project Supervisor: Dr. E. Levis
Team Supervisor: Dr. E. Levis
Submission Date: 15/06/2020

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ

Abstract

This report aims to model the in-air capture of a descending rocket stage by a conceptual remote-piloted synchropter, and investigate the helicopters handling qualities with and without external payload, using a mix of qualitative and quantitative methods. The flight simulator *X-Plane 11* was used along custom developed programs to simulate the capture and handling qualities. A combination of existing and newly developed testing criteria was used to asses the performance of the helicopter.

Although deficiencies were found in the helicopter's sensors, the air-frame was found to be capable of preforming the capture. Excellent handling qualities were exhibited for roll and pitch, which fully met military specifications. However yaw authority was found to be poor in forward flight, and non-existent in hovering flight, and as such did not meet military specifications. Slung load flight testing revealed that the helicopter unable to hover with the heavier payload, but otherwise met the mission criteria. After testing, multiple design recommendations are made, as well as problems with the simulation identified.

WRR Team Members

Table 1: *Team Members and Roles.*

Name	Team Name	Team Code
Alessandro Bullitta Yassine Ghenima Cyril Gliner	Project co-ordination	WRR01
Chi Wing Cheng Alessandro Rossi Polvara Tin Hang Un Le Yin	Aerodynamics	WRR02
Anton Le Tejasva Malhotra Atri Sharma Alastair Wood	Structures & CAD	WRR03
Omar Arangath Thomas Cross Zuzanna Janusz Sam Jenner Jo Tan Tom Walker	Systems & Propulsion	WRR04
Filip Bastien Ananya Dubey Kai San Hon Andrew Ng India Tavares	Flight Mechanics, Performance & Control	WRR05
Tom Alston Hakan Serpen Tze Leung Wong	Flight Simulation & Testing	WRR06

†Sub-team leads are shown in **bold**

Contents

1	Introduction and Pre-Conceptual Work	1
1.1	Pre-Conceptual	1
2	Software Selection	1
2.1	Flight Gear	1
2.2	X-Plane Modelling and Limitations	2
2.3	Data Output	2
3	Capture Modelling	2
3.1	Phase 1: Maneuver	3
3.2	Phase 2: Capture	4
3.3	Phase 3: Slung Load	5
4	Flight Testing	5
4.1	Capture Flight Testing	5
4.1.1	Phase 1	5
4.1.2	Phase 2	6
4.2	Handling Quality	7
4.2.1	Frequency Domain Testing	8
4.2.2	Slung Load	10
5	Discussion and Conclusions	11
5.1	Identified Helicopter Deficiencies	11
5.2	Simulation Deficiencies	12
5.3	Conclusion and Next Steps	12
A	Appendices	14
A.1	Flight-Station Camera Setup	14

A.2 VRS Limits	14
A.3 Frequency Domain	15
A.4 Flight Test Cards	16
B Plugin Code	17

List of Tables

1	<i>Team Members and Roles.</i>	ii
2	<i>Table of Distance Margins and achieved Average Variations</i>	5
3	<i>Slung Load Results</i>	10
4	<i>Transfer Function</i>	15

List of Figures

1	Force Vectors of Helicopter Blade	2
2	Velocity Altitude Relationship of Different Stages [10]	3
3	Rocket Payload in PlaneMaker	3
4	Displacement Time History	4
5	Force Time History ($\zeta = 0.4559, \omega = 50$)	4
6	Distance Graphs Between Helicopter and Payload Reference Point	6
7	Response to Time Varying Force (1.2 Dynamic Load Factor)	7
8	Bode Plot of Roll Response	9
9	Bode Plot of Pitch Response	9
10	Bode Plot of Yaw Response	9
11	VRS Speed and Descent Rate Boundaries [14]	14
12	Level Boundaries during divided attention operations [16]	15
13	Time History for Pitch Response in forward flight	15

1 Introduction and Pre-Conceptual Work

In this group project, a remote piloted synchropter, which could recover descending rocket stages, was created. It was designed to recover a payload of up to 1.4 tons. The objective of the flight simulation team was to accurately simulate the created helicopter, and test its flight characteristics and performance; to ensure it meets regulations and mission requirements. One key aspect to simulate was the aerial capture maneuver, to see if it was possible and if the helicopter could safely achieve it. Another important thing to test was the handling qualities of the the helicopter. Handling qualities in this report refers to the immediate helicopter response to a control input, as opposed to the longer term response to a control input covered in [1], which is more focused on stability as opposed to maneuverability .

1.1 Pre-Conceptual

During the first week of the project a pre-conceptual proposal for using a helicopter as the recovery vehicle was drawn up. The recovery of the rocket will take place over the ocean, and using impact data from *Soyuz* launches [2], It was realised that the helicopter be unable to fly the long distances required, and therefore a boat mother-ship would need to be used. A Boat would sail to the expected recovery site, the helicopter would be launched from it, recover the payload, deposit it on the boat, and then land on the boat. In the initial plan, each boat/helicopter would recover one individual stage.

The most suitable type of ship to use was Multi Purpose Support Vessels. These ships are highly flexible, have significant cargo space and can be chartered for a variety of purposes. To get an initial cost for the boat, estimated operating costs for MPSV's were used [3], assuming a fairly high specification vessel (Due to Helicopter and Cargo Requirements) the daily operating cost was found to be \$40,000. Due to low demand, suppliers operate at or near break-even margins; therefore a small 5% profit was added to find a daily charter cost of \$42,000. Using the the distance data from the Soyuz Impact sites, an estimated boat cost of \$100,000 per component recovered per launch was found.

2 Software Selection

In order to simulate the helicopter, a simulation software had to be chosen. The two options considered were *Flight-Gear* and *X-Plane* both of which have different advantages and disadvantages. Additionally within *Flight Gear* there are two different flight models: *YaSim* and JSBSim. In addition to flight modelling accuracy, the feasibility of modelling the capture also had to be considered.

2.1 Flight Gear

The key advantage of *Flight Gear* is it's flexibility. It is possible to model any kind of flying vehicle, and additional modelling the capture and the slung load dynamics would be relatively easy. The Flight Gear code is also open source, meaning it can fully be understood. The commonly used flight model in Flight Gear is JSBSim. JSBSim works using the stability derivative method; the stability derivatives for the aircraft are specified to the simulator, in order to drive the simulation [4]. These derivatives would need been found by the Flight-Mech team. The big limitation of this is that the flight simulator can no longer validate the model from WRR05, only replicate it. Furthermore there are practically no existing helicopters in *JSBSim*, meaning it would be extremely difficult to make one from scratch. Many examples of helicopters were found to use the *YaSim* flight model, so it was considered next. *YaSim* uses geometric data of the helicopters to define the flight model, similar to

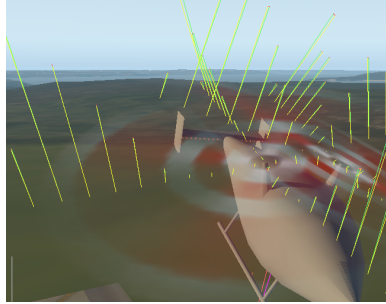


Figure 1: Force Vectors of Helicopter Blade

X-plane. However also requires the user to input scaling factors to make the aircraft match its real performance, which is obviously unsuitable for flight testing [5]. Due to the disadvantages of *Flight Gear*, *X-Plane* was used.

2.2 X-Plane Modelling and Limitations

X-Plane works using the Blade Element Method (BEM), the aircraft is broken down into smaller sections, and velocity components at each section are found. Based on the lift/drag coefficient at that section, a sectional force can be calculated, these forces are added up and accelerations found and the cycle repeats [6]. This process repeats around 20 times a second.

The decision was made to use *X-Plane 11* over *X-Plane 10*, due to improvements in helicopter modelling [7]. Additionally, the new experimental flight model could be used which contains improved and validated down-wash modelling [8].

Helicopter dynamics occur at very high frequency. Even the the blades rotate at up to 10Hz, a frequency that is too high for *X-Plane* to actually simulate. Therefore, instead of modelling a rotating blade, *X-Plane* models it as a disk. Each rotor disk models a force distribution acting along two axes, split into 10 sections for each side. These sections are sections of the blade. This is shown in Figure 1. Changing the collective and cyclic adjusts the angle of attack at each section, and therefore the lift. Although the simulation does have a flapping parameter, as the whole rotor is modelled as a disk, this is likely an empirical constant. Other limitations include only modelling fuselage drag and not lift, and using empirical constants for correcting 3D effects as opposed to a vortex model. Despite these simplifications, *X-Plane* still represents an accurate simulation.

2.3 Data Output

Data is outputted from *X-Plane* using User Datagram Protocol (UDP). UDP packets are sent from *X-plane* to the receiver of the data at over 50 times a second. data such as input commands can also be sent to *X-Plane* through UDP. *SIMULINK* was used for Data output and input due to the direct integration with *MATLAB* for plotting, and the ability to directly implement the controller that WRR05 created.

3 Capture Modelling

One of the objectives of the Simulation team was to try and simulate the entire capture process, to see if how difficult it was, and if it was feasible. As mentioned in Section 2, one of the disadvantages

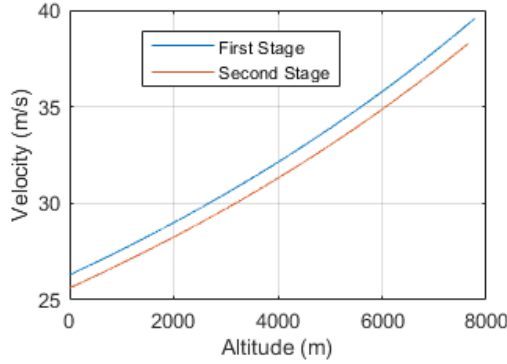


Figure 2: Velocity Altitude Relationship of Different Stages [10]

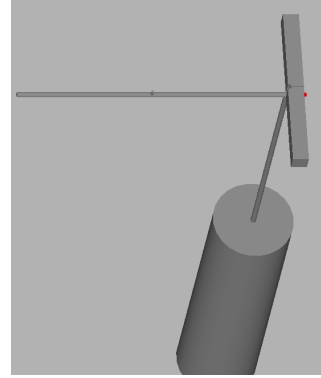


Figure 3: Rocket Payload in PlaneMaker

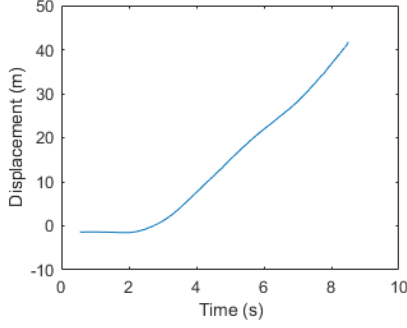
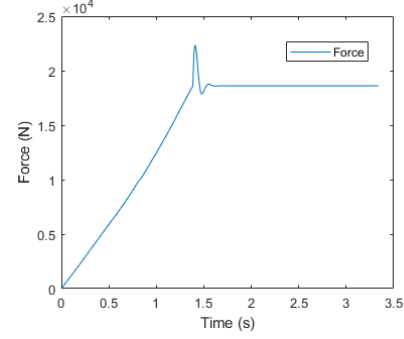
of *X-Plane* is that it is not very flexible, making the process of modelling the capture more difficult. Therefore, to robustly and accurately model the whole process, it would be broken down into three key stages, each with its own distinct modelling strategy. The first phase was the maneuver phase, in this phase, the helicopter would intercept the payload, and attempt to perform the capture maneuver. The second phase was the instantaneous force phase, as helicopter captured the rocket, a dynamic, time varying force would be applied on the helicopter. The final phase was the slung load modelling phase, where the helicopters performance and handling quality with the attached external load was assessed.

3.1 Phase 1: Maneuver

The key objective of this test is mainly to verify whether the capture maneuver is possible, and if so how difficult it is. An obvious limitation of this test is that the author is not the highly-skilled helicopter pilot required for this mission, and as such will find the maneuver significantly more difficult than a real operator would. However, as the helicopter is unmanned, conducting the maneuver in the simulation is much closer to the real thing than if the helicopter was manned; and the associated benefits (eg. 360° camera) and drawbacks (eg. Lack of physical feedback) of unmanned flight can be assessed. Information about the cameras on the helicopter was obtained from WRR04 [9], and implemented. One monitor displayed the forward 95° FOV facing camera, and the other displayed the 48° FOV belly mounted payload camera¹.

The rocket descends to earth with a parafoil, which remains at a constant heading, with a 15° glide-slope during the maneuver. The trajectory of both stages was calculated by the launch team, as shown in Figure 2. To simulate the descending stage in *X-Plane*, an aircraft model of the rocket was made in *PlaneMaker*, and was used as an AI aircraft in the simulation. As the trajectory was already calculated, an actual flight model did not need to be created. As such, a crude 3D model of the rocket was made in *Google SketchUp*, and imported to *X-Plane* as a misc body. The reference point of the payload was set as the middle of the drogue line, and a small bump was added on the 3D model at this point, shown in Figure 3. To make the AI rocket follow the required flight path, a custom plugin needed to be created. In *X-Plane* each time the software does the BEM calculation, it is referred to as a flight loop. The *X-Plane* SDK enables you to run certain functions during each flight loop. The created plugin overwrites the calculated velocity data-refs of the AI model each flight loop, to force it to follow a certain flight-path. As the velocity of the rocket changes with

¹See Appendix For Picture of Setup

**Figure 4:** Displacement Time History**Figure 5:** Force Time History ($\zeta = 0.4559, \omega = 50$)

altitude, a cubic function was fitted onto the WRL team's data [10] to find the rocket velocity as a function of altitude. The constant glide-slope meant that the vertical and horizontal velocity could be found using simple trigonometry. Once the plugin had been completed, the helicopter could attempt the capture manoeuvre with the payload. To validate whether the capture manoeuvre was successful, data-refs of the positions of the rocket and the helicopter were read, and analyzed to see if capture was successful, which will be further expanded upon in Section 4.1.

3.2 Phase 2: Capture

As the helicopter captures the payload, a dynamic, time varying force will act on the helicopter, which could possibly destabilize it, and as such this force needed to be modelled to see what effect it would actually have on the helicopter. When the helicopter captures the payload, a slider at the bottom of the parafoil slides up to deflate the parafoil. Coming up for a model for this force proved to be challenging, as it is a complex system, and there were no existing papers about this topic. In collaboration with WRR04 [11], the decision was made to split the force time history into two parts: The first part being displacement driven (slider sliding up), and the second part being approximated as a 2nd order dynamic system, with specified damping ratio and natural frequency.

The force time history of the displacement driven section is dependent on how quickly the helicopter ascends away from the payload during the capture. To find this out, while following the payload, a climb was performed and the displacement time history (Figure 4) was converted into a force time history (Figure 5) in [11], through considering the length of the parafoil cables. The force was applied at the helicopter winch point, and the worst case angle of the cable was considered when defining the force vector. When defining this worst case angle, it was assumed that the helicopter would have zero roll and pitch.

The implementation of this force is similar to the first phase, during each flight loop, additional forces and moments can be applied to the aircraft. A quadratic curve was fit for the initial displacement-driven phase, for the second phase, the equation for the 2nd order system was coded into the plugin, such that the force could be found at a given time. The force is activated by pressing a button, and a 3 second delay is given such that the pilot is able to regain trim after pressing the button. A complication arose from the fact that the additional forces are defined in the body-fixed axes, but the position of the payload during the maneuver is considered in the global reference frame. To resolve this issue, transformations using Euler angles were applied to the forces, for a given helicopter pitch and roll angle (yaw angle was not considered).

Table 2: *Table of Distance Margins and achieved Average Variations*

	Distance Margin (meters)	Description	Achieved Average Variation (2σ)
Longitudinal	8.18	Slung line must lie within 75% of the drogue line length (From payload end of drogue line) throughout manoeuvre	10.56
Lateral	3.05	When intercepting laterally the helicopter can overlap the drogue line by up to this distance (Overlap must be greater than 0)	2.46
Vertical	5.71	Drogue Line needs to lie within 75% length of the sling line (From the hook) throughout manoeuvre	3.5

3.3 Phase 3: Slung Load

A number of options were considered in how best to model the slung load. *X-Plane 11* has built in sling-load functionality, so this was investigated first. Although there were basic options to change the length of rope, mass and size of the object; there was no way to change more detailed aspects (eg. C_D of object). Most importantly, due to the closed source nature of X-Plane, the capabilities and limitations of the modelling could not be found, as there was no documentation or posts about it on the forums. So other avenues were considered. An idea was explored to create a plugin which modelled the payload as an AI aircraft, and add a force on the the helicopter and payload, to simulate a rope connection. However to properly implement this idea, it would involve knowing the forces acting on the AI aircraft. This information is not available in the datarefs, only the velocity and position data is, so this idea was not implemented. Ultimately, an open source, physics focused external sling load plugin named Helicopter Sling Load was found online and was utilized [12].

The plugin works by solving differential equations based on helicopter and payload position numerically. To increase accuracy, the calculation runs separately from the *X-Plane* physics engine, and at a much higher frequency. The calculated forces are then applied through the same method described previously [13]. The plugin models the payload as a cuboid, which size and coefficient of drag values are adjustable. The rope is modelled as a single mass oscillator, and realistically, only tension forces are applied to the helicopter. However some aspects are not modelled. Down-wash is not modelled on the cargo, however as the sling line over 29m long, the actual influence of down-wash would be minimal anyway. The rotation of payload is also not considered, although as the rocket is a cylinder, it is symmetrical in rotation so this is not major issue.

4 Flight Testing

4.1 Capture Flight Testing

4.1.1 Phase 1

To capture the payload the helicopter needs to hover next to and above the parafoil, laterally intercept the drogue line with the sling line, and then climb. The hook has an aerodynamic component which can change its drag, which enables it to change its longitudinal position. Allowing for pilot error the following acceptability margins were defined in Table 2: The test started with the helicopter near the payload at 20,000ft. The procedure is outlined in the flight test card (A.4), and was attempted several times, but due to pilot inexperience, the maneuver could not be completed

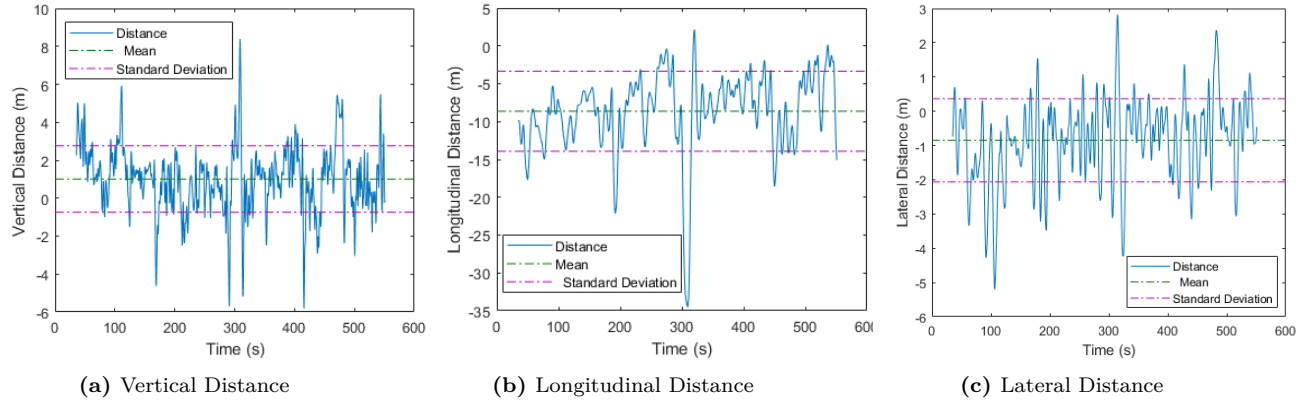


Figure 6: Distance Graphs Between Helicopter and Payload Reference Point

successfully. Problems also existed with the camera setup, the belly camera's field of view was too narrow, so it was difficult to see the payload. Furthermore when the helicopter pitches and rolls, the belly camera view also changes, making it useless in helping the pilot know his relative position to the payload when above it.

Although deficiencies in the camera setup had been identified, it was still not clear if the helicopter was capable of performing the maneuver. Therefore a new test was devised, where the pilot would attempt to follow a reference point on the payload, and a time history of the longitudinal, lateral and vertical distance between the helicopter and the payload would be plotted. These are shown in Figure 6. The mean distance along with the standard deviation was found in each case. The distance margins were compared to twice the standard deviation, which was considered the average variation of distance between the helicopter and reference point, and is shown in Table 2.

The vertical distance was easy to maintain, the the large collective range meaning that the helicopter could comfortably match the high descent rate of the payload, and average variation sat comfortably within vertical margin limits. This was also the case for the lateral distance, it was easy to stay close as the lateral position of the payload did not change. However following the longitudinal position proved to be challenging. The average variation was over the maximum distance margin, and the mean longitudinal distance of 8.65m is very poor. Unlike changing vertical speed, there is a delay from a pitch control input to a change in horizontal speed, and overshooting was common and problematic. Furthermore, the payload changes speed throughout the descent, meaning that trying to match the speed of the helicopter was difficult and could only be done through visual reference, which was ultimately inadequate. These problems however are due to low pilot skill, and ultimately the air-frame itself is more than capable of performing the maneuver.

4.1.2 Phase 2

The purpose of testing the instantaneous force response is to make sure the helicopter remains controllable and is not dramatically displaced. Another potential risk is that the helicopter is slowed down enough, and is forced to descend quickly enough that it could transition into vortex ring state. Vortex ring state is a dangerous condition which occurs when air flows up at the hub of the rotor (due to the descent), and the induced velocity forces causes blade stall, leading to a loss of lift and control. So the vertical and indicated airspeed during the capture was also looked at to make sure it did not exceed VRS limits.

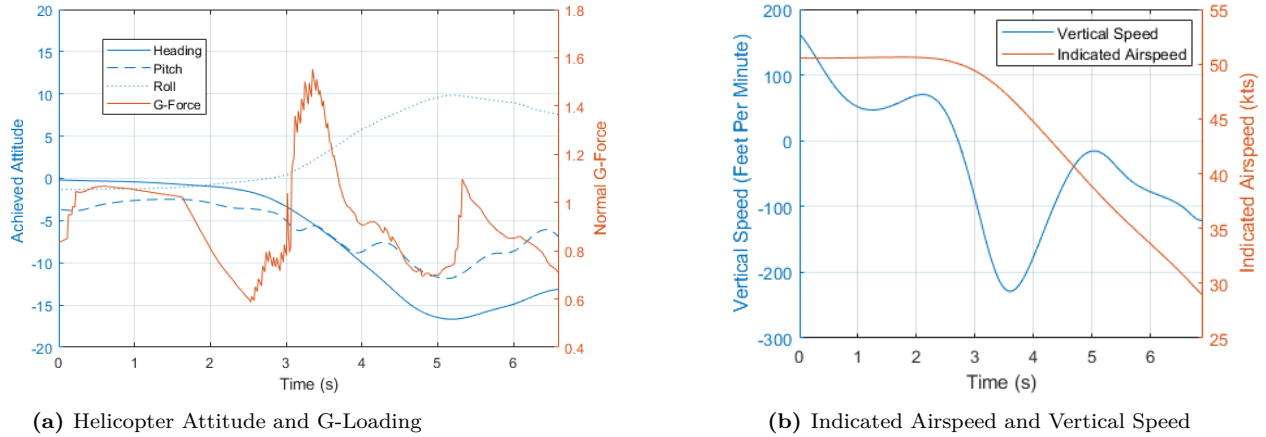


Figure 7: Response to Time Varying Force (1.2 Dynamic Load Factor)

When trying to decide the level of force that was going to be applied to the aircraft, it was found the maximum dynamic load observed during similar maneuvers was around 1.2 times the static load [15]. The static load for this test was based on the the mass of the first stage, as well as the drag that would act on it at 30m/s. The displacement driven force was designed so it would finish at the static load, and the damping ratio of the 2nd order system was tuned so that the maximum force in the time history would be 1.2 times the static load. The natural frequency was set at 50 rad/s, and the test took place at 15,000ft. The lateral angle for the force vector was based on the maximum allowed lateral overlap in Table 2 and the length of the sling line, and was 20°. The longitudinal angle was set to 34°. During the maneuver the pilot would attempt to keep the helicopter in a stable condition, and prevent the speed and descent rate from getting too low.

The attitude is plotted against the normal body axis g-loading of the helicopter in Figure 7a. The initial reduction in g-loading can be attributed to the displacement driven section, and the subsequent spikes due to sharp collective increases which result from an attempt to reduce descent rate and the 2nd part of the loading. From a pilots perspective, the transition from a steadily increasing force, to the damped oscillatory force is challenging, even when prepared for it. In terms of attitude, the force had the greatest effect on the heading, with a maximum change of over 15°, most likely due to the helicopters poor yaw authority. Although there was a significant change in attitude, it was soon recovered which indicates the helicopter is capable of handling the force.

The vertical descent speed increases as the force is applied, shown in Figure 7. This vertical descent speed does not approach the the VRS limits (Shown in A.2 [14]), although it continues to slowly increase over time. The indicated airspeed decreases with time, which combined with the increased vertical speed could eventually lead to a VRS state. A significant limitation in the force modelling is that the steady state force was calculated considering the drag of the payload. As the speed decreases, in real life the drag force, and therefore the steady state force will decrease. However this is not the case for the plugin, and the force is overestimated.

4.2 Handling Quality

The ADS-33E [16], the military rotor-craft handling specification was used to benchmark the handling quality of the helicopter. The specification uses a simplified Cooper Harper Handling Qualities Rating System: Level 1 corresponds to good/fair handling qualities, Level 2 indicates

poor, but acceptable handling qualities, and Level 3 Handling qualities are considered very bad, and unacceptable. As the handling quality is based on the perception of the pilot, it is fairly qualitative and somewhat subjective, so an effort was made to quantify the handling quality using frequency domain testing.

4.2.1 Frequency Domain Testing

Frequency domain testing involves testing the response of the helicopter to control inputs of different frequency. It is heavily featured in ADS-33E, and is very important in defining helicopter handling qualities. The frequency response for roll, pitch and yaw was found at forward flight and hover, without the presence of an external load.

To perform the test, the pilot conducts sweeps of increasing frequency in one control axis, and the time history of the control input and achieved attitude is found (Example Shown In A.3). Originally a signal generator was used to output a sinusoidal control input of increasing frequency from *Simulink* to *X-Plane* as suggested by [17]. However as it was difficult to get the helicopter into a good trim, the sinusoidal input ended up destabilizing the helicopter, so ultimately was not used. The time response information is de-trended, and then converted to the frequency domain using a Fast Fourier transform².

A bode plot of the response (output/input) is created. The expected trend is that as the frequency of the input increases, the magnitude of the response decreases, and becomes more out of phase with the input. Two key parameters called Aircraft Bandwidth and Phase Delay are found from the bode plot. Bandwidth is defined as ω_{-135} . The phase delay is found using Equation 1 [18].

$$\tau_p = -\frac{\phi_{2\omega_{180}} + 180}{114.6 \omega_{180}} \quad (1)$$

$\phi_{2\omega_{180}}$ is the phase at twice the ω_{180} . At this frequency, the data is expected to be too noisy and incoherent to measure. Therefore a linear fit is found of the phase plot near ω_{180} and extrapolated to $2\omega_{180}$.

A lower bandwidth indicates a helicopter which is less responsive to higher frequency control inputs, and has worse handling qualities. A low phase delay reduces the tolerance for low bandwidth [19]. Based on the bandwidth and phase delay, a handling quality level can be defined for a given response and mission requirement [16] (See Appendix). For this report the level boundaries were taken for cargo helicopters for divided attention operations (Due to difficulty in catching rocket.)

Roll and Pitch Response Roll Tests were conducted at hover and 30kts at 3,000 ft, the response for roll is shown in Figure 8. The response for hover and forward flight is similar which is to be expected [18]. The amplitude starts high and gradually rolls as the frequency increases. The bandwidth was 16.4 rad/s for both hover and forward flight, the phase delay being 0.0615 for forward flight, and 0.089 for hover. The bandwidth is very high, this makes sense as the aircraft is extremely responsive in roll. The difference in phase delay was slightly noticeable for the pilot, as there was a slight increase in delay between the control input and the output, however this

²In modern studies, normally a Chirp-Z transform is used instead of a Fast Fourier Transform due to increased accuracy. However in order to take advantage of the *MATLAB System Identification Toolbox* (which doesn't have Chirp-Z transforms) a Fast Fourier Transform was used instead

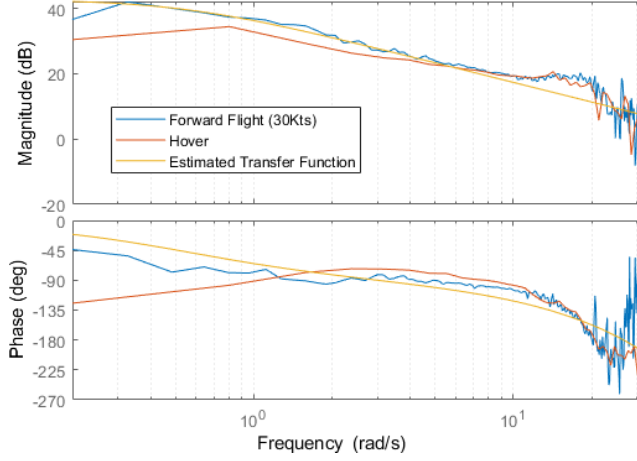


Figure 8: Bode Plot of Roll Response

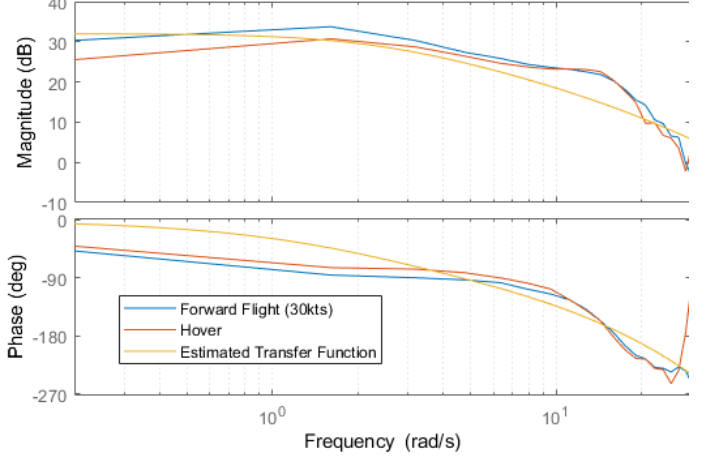


Figure 9: Bode Plot of Pitch Response

difference was slight. The roll response easily meets the ADS-33E military specification for Level 1 handling qualities.

Pitch tests were conducted at hover and 30kts at 3,000 ft, and the results (Figure 9) were similar to the roll response, with very responsive performance in both hover and forward flight. The bandwidth and phase were slightly worse than the roll response, forward flight had a bandwidth of 12.2 rad/s and a phase delay of -0.0961. Hover flight a bandwidth of 12.3 rad/s and a phase delay of -0.1036. However these were still well within Level 1 handling quality.

Estimated Transfer functions were also fitted to the responses and are listed in the appendix (A.3). Based on [18], the transfer function modelled as a 2nd order equation with time delay, and with two poles and no zeros. Theoretically the time delay of the transfer function should correspond to the phase delay although this is often not the case.

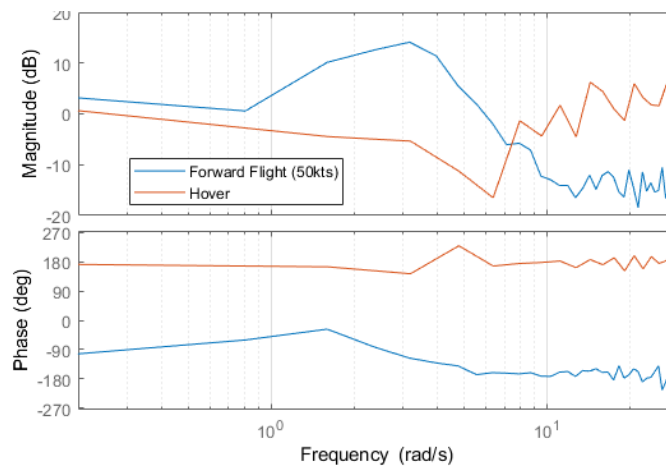


Figure 10: Bode Plot of Yaw Response

Yaw Response Due to the difficulty in yawing at lower speeds, the yaw response was tested at 50kts for forward flight to get clearer results. The difference of the pitch and roll response, compared

Table 3: *Slung Load Results*

Speed	Stage 1		Stage 2	
	Handling Level	Trail Angle (°)	Handling Level	Trail Angle (°)
0kts	Level 3	-	Level 2	-
10kts	Level 2	-	Level 1	-
20kts	Level 1	3.8	Level 1	4.6
30kts	Level 1	9	Level 1	8.5
40kts	Level 1	13	Level 2	12
50kts	Level 1	20	Level 2	18
60kts	Level 2	24.5	Level 3	22
70kts	Level 2	27.8	Level 3	27

with the yaw response is clearly shown in Figure 10. Starting with the forward flight, the magnitude is significantly lower than the other bode plots. A large yaw input was required to get a substantial heading change, which is the reason for this. The phase delay for forward flight was 0.061, and the bandwidth was 3.6 rad/s. The bandwidth was much lower than the other two responses, suggesting yaw control was more sluggish, which is indicative of the actual piloting experience. Nevertheless it still meets the Level 1 handling qualities requirements. It was nearly impossible to control the yaw during hover. This is reflected in the bode plot: The magnitude stays below zero for most of the frequency range; as the joystick input only goes from -1 to 1, this indicates a less than 1 degree change in achieved heading change which is negligible. The phase stays around 180°, which does not match at all with expected results, suggesting the results are invalid. For this reason, the bandwidth and phase delay were not found and the yaw response in hover is considered to have a handling quality of Level 3.

4.2.2 Slung Load

To determine how to test handling quality with a slung load a NASA report [20] and the ADS-33E [16] was consulted, which both suggested that handling quality should be found based on qualitative test procedure. The key element of testing is to develop a flight envelope of safe speeds at which the slung load can be flown. The test procedure is to fly steadily at increasing speeds and make small displacements to the controls, and observe the reaction of the load and the helicopter, and assign a handling quality. Once a maximum safe speed had been determined, maneuvers up to and including a 30° bank would be conducted, to see if the load still has the same handling quality, and if not it is overwritten [20]. The trail angle is the angle at which the load trails behind the helicopter in steady level flight, relative to the normal axis. This trail angle was also found, to make sure it was not too excessive (below 45°), such that it could potentially hit the helicopter. The results are summarised in Table 3 for the two different rocket stages. Tests were conducted at 3,000ft.

Starting with the 1st stage, the helicopter was unable to hover with the payload, and slowly descends. Due to the fact that ascent is impossible, and the high risk of a getting into a VRS state, this speed was rated Level 3. The helicopter is able to fly with the payload at 10kts, but is classified at Level 2 as it is very easy to inadvertently slow down to the dangerous Level 3 speed. The load and helicopter are stable for the rest of the speeds in forward flight, although significant oscillation occurs when manoeuvring above 60kts, so it receives the Level 2 classification from this point. With the 1st stage, the helicopter reached a maximum speed of 73kts.

For the lighter 2nd stage, the helicopter was able to hover with the payload. Due to the non-existent yaw control, when the payload began to oscillate it was difficult to control the heading of the helicopter although it was possible with roll control, so it was given a Level 2 handling quality. Acceptable handling qualities were experienced past this speed until 40kts. From this speed, the load oscillates excessively, although the helicopter is not dramatically effected and is still given a Level 2 rating due to the oscillation. From 60kts, the load oscillates at angles of over 45°, when banking, which is unacceptable, and therefore given a Level 3 Handling quality. Although the helicopter is capable of exceeding 70kts with the smaller payload, this was not tested as it was already found to be unsafe at 60kts.

The trail angle for both loads were very similar, and always remained below 45°. In the mission profile, the helicopter is supposed to cruise with payload at 54kts. The helicopter is just capable of meeting this requirement. However, the fact that it is unable to hover with the 1st stage is very problematic for recovery, the only way to mitigate this is to have the recovery ship sail into the wind. The handling quality results was compared to the cargo's root-locus diagram created by WRR05 [21] (1st Stage), which found the stability of the payload at 20,30,and 40m/s. It predicts that the oscillations become more damped at 30m/s than 20m/s and less damped at 40m/s. This does not match up fully with the simulation as it was the oscillations general found to get larger as the flight speed increases.

5 Discussion and Conclusions

5.1 Identified Helicopter Deficiencies

After flight testing, the results are analysed and deficiencies in the design identified. From Phase 1 capture flight testing, it was found that the camera setup was insufficient. The downwards facing payload camera field of view was too narrow, and moved with the roll of the helicopter. A recommendation is made to replace the camera with one which has a wider field of view and is ground stabilized. The workload is also too high for a single pilot, so multiple pilots/spotters are required.

Handling quality for pitch and roll was excellent, if not too responsive, which could easily be solved by decreasing joystick sensitivity. The Helicopter has an all moving horizontal stabilizer, to increase pitch authority in forward flight. However, the pitch response was found to be very similar in forward and hover flight. This denotes that that the moving stabilizer has little to no effect, and therefore should be scrapped to save weight and cost.

Handling quality for yaw was very different. In high speed forward flight, thanks to the rudder, yaw control was possible, but heavily inferior to pitch and roll control. However at low/hover speeds, the rudder has no effect. The yaw control of the helicopter is supposed to be achieved through differential cyclic for each rotor. The tunings for the differential cyclic were done manually, and may have been incorrect. However other syncopters, such as the K-1200 K-MAX, use differential *collective* as well as differential cyclic to control heading [22]; and subsequent flight testing showed improved yaw authority at low speed when differential collective was added.

The other major deficiency in the helicopter is the fact it was unable to hover with full payload. The reason the helicopter can fly forward with payload is due to the fact that vortices around the rotor get swept away, and the induced velocity of the blade is reduced, which increases lift. This effect is known as transitional lift. The reason for this inadequate hover performance was investigated.

Originally the engine was thought to be under-powered, however the engine horsepower rarely came close to maximum throughout the flight, and although there was a drop in rotor rpm when hovering with the payload, the rotor rpm actually remained above the design rpm, which suggests the problem is with the blade design not producing enough lift. This was confirmed when looking at the force vectors in *X-Plane* and can be seen in Figure 1. The force generated by the blade should increase along the blade, however the force at the tip of the blade is actually lower. The reason for this is discussed in the following section

5.2 Simulation Deficiencies

When the blade was designed in the *PlaneMaker*, due to difficulty in interpolating the root and tip airfoils³, one airfoil was used for the entire section. However the twist distribution was not changed for the tip section. As a result, the blade was too heavily twisted for the airfoil used, which caused the reduction in lift at the tip and a loss of performance.

Another limitation of the simulation occurred when testing the instantaneous force. The force acts on a very short time scale (around 5 seconds), however the simulation only calculates forces at a 20Hz frequency. This meant that the simulated force, especially in the 2nd order section, was fairly inaccurate. Additionally increasing the frequency of the 2nd order force caused the simulation to lag, so it had to be kept at unrealistically low levels. The same limitation applies to the sling load plugin. Although it calculates helicopter/cargo positions and forces at a much higher frequency it still only applies the forces every flight loop. This sometimes led to noticeable lag, and occasional bugs.

The frequencies of interest in the frequency domain test are almost an order of magnitude smaller than the frequency of *X-Plane* runs at, so in theory the results should be valid. However there are potentially frequency modes of the helicopter structure/rotor, which could be excited by a certain frequency input, such as a rotor-pylon mode, as described in [18], which would influence the response. These frequency effects are not modeled, and therefore not observed in *X-Plane*. Finally, cyclic and collective angles were not properly modelled. The design calls for a maximum blade pitch of 23°, which is inclusive of collective *and* cyclic. However in *X-Plane* these angles are decoupled, such that collective pitch went up to 23° with up to additional 12° of cyclic on top of that, leading to an overestimation of control authority at higher collective angles.

5.3 Conclusion and Next Steps

The helicopter was able to successfully meet most of the mission requirements and military specification. The key identified deficiencies were in yaw authority, especially in low speeds, and a lack of sufficient blade lift at the tip (although this problem was likely a fault of improper simulation). With more time, potential next steps would be to conduct frequency domain analysis with the slung load as well, and compare compare the fitted transfer functions with the model created by WRR05. Additional force tests with a greater load factor could be conducted, to test the limits of what the syncopter is capable of. Finally the issues with airfoil interpolation worked on and resolved, and improved differential gains tested for yaw control.

³The difficulty came from the fact the the airfoil only starts changing at 90% distance along the blade

References

- [1] Alston,T. Flight Simulation and Testing of a Novel Synchropter Concept for the Aerial-Recovery of Falling Rocket Stages. WRR Group Design Project, Flight Simulation and Testing. 2020
- [2] Zak A. How Soyuz rides into orbit [Internet]. Russianspaceweb.com. 2019 [cited 15 June 2020]. Available from: http://www.russianspaceweb.com/soyuz_launch.html
- [3] Multi-purpose Support Vessels [Internet]. Daleel Supply Chain Management Network. 2015 [cited 15 June 2020]. Available from: <https://www.scmdaleel.com/category/multi-purpose-support-vessels/209>
- [4] Berndt,J. JSBSim Reference Manual. 2011
- [5] YASim - FlightGear wiki [Internet] [cited 15 June 2020]. Available from: <http://wiki.flightgear.org/YASim>.
- [6] How X-Plane Works, X-Plane, [Internet] [cited 15 June 2020], Available from: <https://www.x-plane.com/desktop/how-x-plane-works/>
- [7] Flight Model Improvements Done for X-Plane 11.00 [Internet]. X-Plane Developer. 2016 [cited 15 June 2020]. Available from: <https://developer.x-plane.com/2016/12/flight-model-improvements-done-for-x-plane-11-00/>
- [8] Experimental Flight Model Changes in X-Plane 11.40 [Internet]. X-Plane Developer. 2019 [cited 15 June 2020]. Available from: <https://developer.x-plane.com/2019/09/experimental-flight-model-changes-in-x-plane-11-40/>
- [9] Walker,T. Subsystem Layout of a Novel Synchropter Concept for the Aerial-Recovery of Falling Rocket Stages. WRR Group Design Project, Systems and Propulsion. 2020
- [10] Jablonski, P. Asset Recovery Hardware Design of an Orbital Launch System. WRL Group Design Project, Ground Systems. 2020
- [11] Jenner,S. Capture Architecture Design for the Complete Aerial Recovery of an Orbital Launch System. WRR Group Design Project, Systems and Propulsion. 2020
- [12] Helicopter Sling Load [Internet]. X-Plane.Org Forum. 2020 [cited 15 June 2020]. Available from: <https://forums.x-plane.org/index.php?/files/file/61205-helicopter-sling-load/>
- [13] Helicopter Sling Load Plugin Documentation. 2020 (Available from: <https://github.com/kristian80/HSL>)
- [14] Drees, J., and Hendal, W. Airflow Patterns in the Neighborhood of Helicopter Rotors. Journal of Aircraft Engineering, Vol. 23, (26), 1951, pp. 107–111.
- [15] Jorgensen D, Haggard R, Brown G. The Past, Present, and Future of Mid-Air Retrieval. 18th AIAA Aerodynamic Decelerator Systems Technology Conference and Seminar. 2005.
- [16] ADS-33E-PRF, AERONAUTICAL DESIGN STANDARD PERFORMANCE SPECIFICATION: HANDLING QUALITIES REQUIREMENTS FOR MILITARY ROTORCRAFT (21-MAR-2000)
- [17] Adiprawita W, Ahmad A, Sembiring J. Automated flight test and system identification for rotary wing small aerial platform using frequency responses analysis. International Conference on Intelligent Unmanned System. 2007.

- [18] Tischler, Mark B. Demonstration of frequency-sweep testing technique using a Bell 214-ST helicopter. NASA Technical Memorandum. 1982
- [19] Tyson, Peter H. Simulation validation and flight prediction of UH-60A Black Hawk helicopter/slung load characteristics. Calhoun. 1999
- [20] Key, David L. Airworthiness Qualification Criteria for Rotorcraft with External Sling Loads. NASA Technical Report. 2002.
- [21] Dubey,A. Flight Mechanics Modelling of a Novel Synchropter Concept for the Aerial-Recovery of Falling Rocket Stages. WRR Group Design Project, Flight Mechanics and Control. 2020
- [22] Head E. Going Solo - Vertical Magazine [Internet]. Vertical Magazine. 2015 [cited 15 June 2020]. Available from: <https://www.verticalmag.com/features/goingsolo/>

A Appendices

A.1 Flight-Station Camera Setup



A.2 VRS Limits

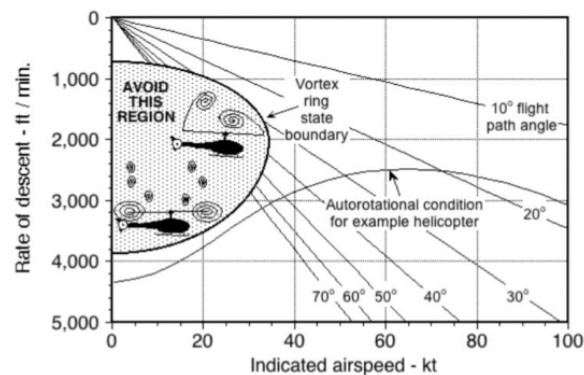
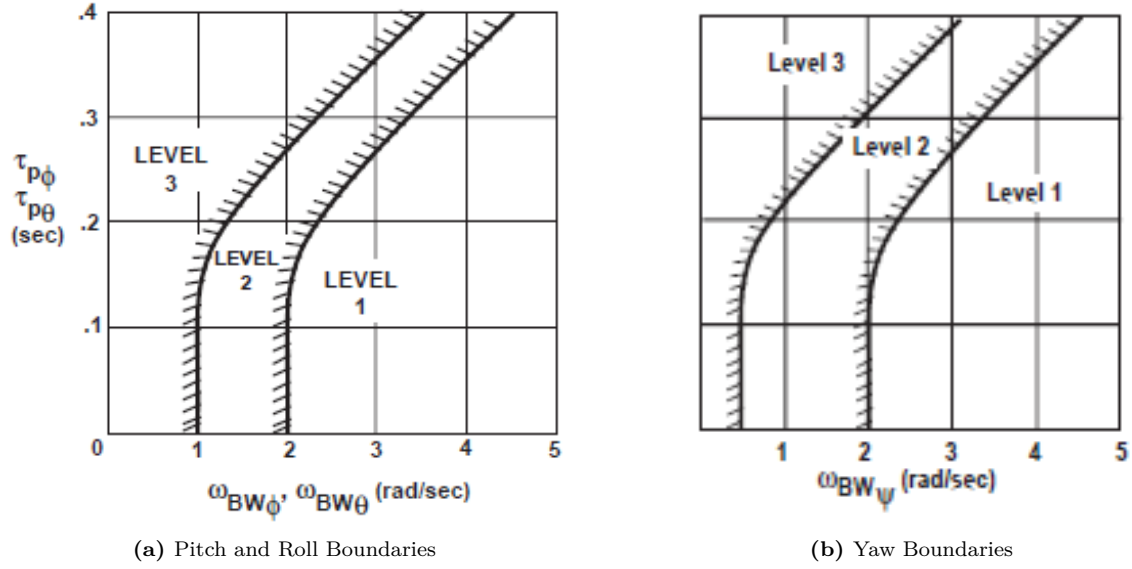
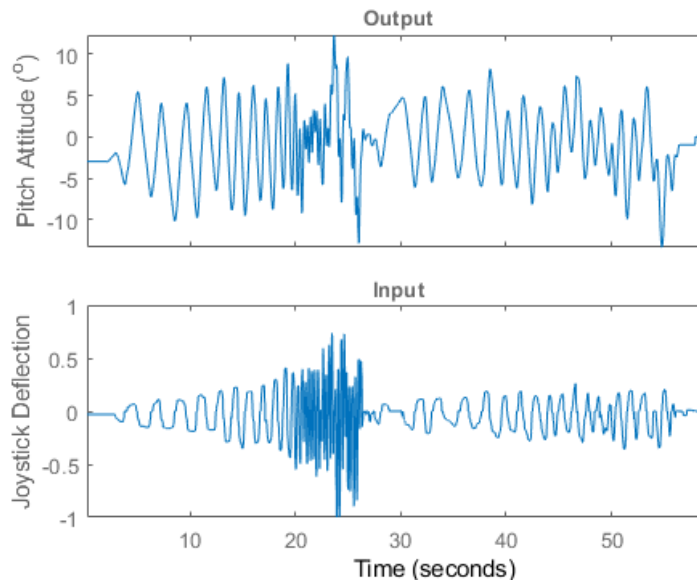


Figure 11: VRS Speed and Descent Rate Boundaries [14]

Table 4: *Transfer Function*

	Transfer Function	Percentage Fit to Data
Roll	$\frac{8.31 \times 10^5 * e^{-0.06}}{s^2 + 1.13 \times 10^4 s + 6.12 \times 10^3}$	55%
Pitch	$\frac{2.23 \times 10^3 * e^{-0.06}}{s^2 + 26.2s + 55.9}$	37%

A.3 Frequency Domain

**Figure 12:** Level Boundaries during divided attention operations [16]**Figure 13:** Time History for Pitch Response in forward flight

A.4 Flight Test Cards

Card No.		Phase 1 Test		
Pilot	Hakan Serpen	Date & Time		
Objectives				
Attempt Capture Manoeuvre				
Configurations				
Stage 1, 20,000ft				
Procedure				Check
Approach Helicopter from behind				
Get within longitudinal and vertical margins and approx. 50ft lateral distance. Hold for 5 Seconds				
Move helicopter over drogue line, maintaining existing margins, and between 0-10 overlap with drogue line, Hold for 2 Seconds				
Gentle Ascent. If the manoeuvre seems to be successful, note down simulation time to aid later data analysis, and repeat procedure.				
Data Recording				
Potential Successful capture	Time of Capture			
1				
2				
3				
4				
5				
6				

Card No.		Slung Load Test (Stage 1)		
Pilot	Hakan Serpen	Date & Time		
Objectives				
Characterize Handling Quality and Trail angle of Slung Load				
Configurations				
Stage 1, 3000ft				
Procedure				Check
Get <u>Into</u> Steady Level Flight at Test Speed				
Note down Trail Angle				
Make small adjustments to controls, note handling level.				
After completed, test handling quality at different speeds during and after a 30-degree bank turn, note down new handling quality at each speed. (Up to Max Safe Speed)				
Data Recording				
Test No.	Trail Angle	Handling Quality Level Forward Flight	Handling Quality Level Manoeuvre	
0kts				
10kts				
20kts				
30kts				
40kts				
50kts				

B Plugin Code

Plugin Code used for Phase 1 and Phase 2 of capture

Listing 1: Plugin Code for Payload Trajectory

```

1
2 #include "XPLMDisplay.h"
3 #include "XPLMGraphics.h"
4 #include "XPLMDisplay.h"

```

```

5  #include "XPLMUtilities.h"
6  #include "XPLMDataAccess.h"
7  #include "XPLMProcessing.h"
8  #include "XPLMPlanes.h"
9  #include <string.h>
10 #include <iostream>
11 #include <string>
12 #include <cmath>
13 #if IBM
14 #include <windows.h>
15 #endif
16 #if LIN
17 #include <GL/gl.h>
18 #elif __GNUC__
19 #include <OpenGL/gl.h>
20 #else
21 #include <GL/gl.h>
22 #endif
23
24 #ifndef XPLM300
25 #error This is made to be compiled against the XPLM300 SDK
26 #endif
27
28 //HotKey
29 static XPLMHotKeyID gHotKey = NULL;
30 static void MyHotKeyCallback(void * inRefcon);
31
32 //FlightLoop
33 static float      MyFlightLoopCallback(
34     float          inElapsedSinceLastCall,
35     float          inElapsedTimeSinceLastFlightLoop,
36     int            inCounter,
37     void *         inRefcon);
38 XPLMFlightLoopID floopid = NULL;
39
40 //Timing
41 int count = 0;
42 float time0 = 0;
43 float time = 0;
44 XPLMDataRef timeid = XPLMFindDataRef("sim/time/total_flight_time_sec");
45 std::string countstr;
46 const float pi = 3.14159265359;
47
48 //dATAREFS
49 //AI
50 XPLMDataRef xid = XPLMFindDataRef("sim/multiplayer/position/planel.x");
51 XPLMDataRef yid = XPLMFindDataRef("sim/multiplayer/position/planel.y");
52 XPLMDataRef zid = XPLMFindDataRef("sim/multiplayer/position/planel.z");
53 XPLMDataRef vxid = XPLMFindDataRef("sim/multiplayer/position/planel.v.x");
54 XPLMDataRef vyid = XPLMFindDataRef("sim/multiplayer/position/planel.v.y");
55 XPLMDataRef vzid = XPLMFindDataRef("sim/multiplayer/position/planel.v.z");
56 XPLMDataRef aglid = XPLMFindDataRef("sim/multiplayer/position/planel.el");
57 XPLMDataRef psid = XPLMFindDataRef("sim/multiplayer/position/planel.psi");
58 XPLMDataRef thetaid = XPLMFindDataRef("sim/multiplayer/position/planel.the");
59 XPLMDataRef phid = XPLMFindDataRef("sim/multiplayer/position/planel.phi");
60 //Player
61 XPLMDataRef pxid = XPLMFindDataRef(" sim/multiplayer/position/local");
62 XPLMDataRef pyid = XPLMFindDataRef("sim/flightmodel/position/local.y");
63 XPLMDataRef pzid = XPLMFindDataRef("sim/flightmodel/position/local.z");

```

```

64 XPLMDataRef pvxid = XPLMFindDataRef("sim/flightmodel/position/local_vx");
65 XPLMDataRef pvxid = XPLMFindDataRef("sim/flightmodel/position/local_vy");
66 XPLMDataRef pvzid = XPLMFindDataRef("sim/flightmodel/position/local_vz");
67 XPLMDataRef paglid = XPLMFindDataRef("sim/flightmodel/position/y_agl");
68 XPLMDataRef ppsid = XPLMFindDataRef("sim/flightmodel/position/psi");
69 XPLMDataRef pthetaid = XPLMFindDataRef("sim/flightmodel/position/theta");
70 XPLMDataRef pphid = XPLMFindDataRef("sim/flightmodel/position/phi");
71
72 // Initializing Vairables
73 double V = 0;
74 double alt = 0;
75 float vx = 0;
76 float vy = 0;
77 float vz = 0;
78
79 //Payload Trajectory Calculation
80 void speedcalc() {
81     alt = XPLMGetDataf(aglid);
82     V = (pow(alt, 3)*2.22591485182242e-12) + (pow(alt, 2)*3.94620905456984e-08) +
83
84     (alt*0.00126496448830393) + 26.2941758548717;
85     vx = V * cos(15 * pi / 180);
86     vy = -V * sin(15 * pi / 180);
87     XPLMSetDataf(vxid, vx);
88     XPLMSetDataf(vxid, vy);
89     XPLMSetDataf(vzid, vz);
90     XPLMSetDataf(psid, 90);
91     XPLMSetDataf(thetaid, 180);
92     XPLMSetDataf(pphid, 0);
93 };
94
95 PLUGIN_API int XPluginStart(
96     char *      outName,
97     char *      outSig,
98     char *      outDesc)
99 {
100     strcpy(outName, "Phase1");
101     strcpy(outSig, "xpsdk.examples");
102     strcpy(outDesc, "Scenario Begin");
103
104     //HotKey
105     gHotKey = XPLMRegisterHotKey(XPLM-VK.F2, xplm.DownFlag,
106         "Applies Force",
107         MyHotKeyCallback,
108         NULL);
109
110     return 1;
111 }
112
113
114 PLUGIN_API void XPluginStop(void)
115 {
116     XPLMUnregisterHotKey(gHotKey);
117     XPLMDestroyFlightLoop(floopid);
118 }
119
120 PLUGIN_API void XPluginDisable(void) { }
121 PLUGIN_API int XPluginEnable(void) { return 1; }
122 PLUGIN_API void XPluginReceiveMessage(XPLMPluginID inFrom, int inMsg, void * ...

```

```

        inParam) { }
123
124 //Flight loop Function
125 float MyFlightLoopCallback(
126     float          inElapsedSinceLastCall,
127     float          inElapsedTimeSinceLastFlightLoop,
128     int            inCounter,
129     void *         inRefcon)
130 {
131     speedcalc();
132     //-1 to indicate we want function to run again on next flight loop
133     return -1;
134 }
135
136 //When Hotkey is pressed Do this:
137 void MyHotKeyCallback(void* inRefcon) {
138     XPLMSpeakString("IWORK");
139     XPLMSetDataf(xid, 0);
140     XPLMSetDataf(yid, 6250);
141     XPLMSetDataf(zid, 0);
142     XPLMSetDataf(pxid, 500); //Places Player Near AI aircraft
143     XPLMSetDataf(pyid, 6200);
144     XPLMSetDataf(pzid, 0);
145     XPLMSetDataf(pvxid, 0);
146
147     //Creating and scheduling flight loop function
148     XPLMCreateFlightLoop_t floop = { sizeof(XPLMCreateFlightLoop_t),
149     xplm.FlightLoop_Phase_AfterFlightModel, MyFlightLoopCallback, nullptr };
150     floopid = XPLMCreateFlightLoop(&floop);
151     XPLMScheduleFlightLoop(floopid, 0.1, 1);
152
153 }

```

Listing 2: Plugin Code for Instantaneous Force

```

1  #include "XPLMDisplay.h"
2  #include "XPLMGraphics.h"
3  #include "XPLMDisplay.h"
4  #include "XPLMUtilities.h"
5  #include "XPLMDataAccess.h"
6  #include "XPLMProcessing.h"
7  #include <string.h>
8  #include <iostream>
9  #include <string>
10 #include <cmath>
11 #if IBM
12     #include <windows.h>
13 #endif
14 #if LIN
15     #include <GL/gl.h>
16 #elif __GNUC__
17     #include <OpenGL/gl.h>
18 #else
19     #include <GL/gl.h>
20 #endif
21
22 #ifndef XPLM300
23     #error This is made to be compiled against the XPLM300 SDK

```

```

24 #endif
25
26 //HotKey
27 static XPLMHotKeyID gHotKey = NULL;
28 static void MyHotKeyCallback(void * inRefcon);
29
30 //FlightLoop
31 static float      MyFlightLoopCallback(
32     float          inElapsedSinceLastCall,
33     float          inElapsedTimeSinceLastFlightLoop,
34     int            inCounter,
35     void *         inRefcon);
36 XPLMFlightLoopID floopid = NULL;
37
38 const float pi = 3.14159265359;
39 //Timing
40 int count = 0;
41 float time0 = 0;
42 float time = 0;
43 XPLMDataRef timeid = XPLMFindDataRef("sim/time/total-flight-time-sec");
44 std::string countstr;
45
46 //Forces and Angles
47 XPLMDataRef fsidid = XPLMFindDataRef("sim/flightmodel/forces/fside-plugin.acf");
48 XPLMDataRef fnrmid = XPLMFindDataRef("sim/flightmodel/forces/fnrml-plugin.acf");
49 XPLMDataRef faxilid = XPLMFindDataRef("sim/flightmodel/forces/faxil-plugin.acf");
50 XPLMDataRef Lplgid = XPLMFindDataRef("sim/flightmodel/forces/L-plugin.acf");
51 XPLMDataRef Mplgid = XPLMFindDataRef("sim/flightmodel/forces/M-plugin.acf");
52 XPLMDataRef Nplgid = XPLMFindDataRef("sim/flightmodel/forces/N-plugin.acf");
53 XPLMDataRef thetaid = XPLMFindDataRef("sim/flightmodel/position/theta");
54 XPLMDataRef phid = XPLMFindDataRef("sim/flightmodel/position/phi");
55
56 float X = 0; //Rightmost distance from centreline
57 float Y = -0.753; //Distance Above CG
58 float Z = 0.024; //Longitudinal Distance From CG
59
60 float dr = 0.4559; //Damping Ratio
61 float w = 50; //Natural Frequency
62
63 float timedelay = 0;
64 float tforcechange = 1.385; //Time at which forces changes from
65
66 float F1 = 100;
67 float F2 = 0;
68 float latangle = 22; //Angle from Helicopter (Left=Positive)
69 float longangle = 34; //Trail Angle
70 float xVmulti = 0;
71 float yVmulti = 0;
72 float zVmulti = 0;
73
74 float Fx = 0; //Body Side Force
75 float Fy = 0; //Body Normal Force
76 float Fz = 0; //Body Axial Force
77
78 float fside = 0;
79 float L = 0;
80 float fnrml = 0;
81 float M = 0;
82 float faxil = 0;

```

```

83 float N = 0;
84
85 //Calculating Force Vector Using Slung Line angle, and Euler angles to transform ...
    into body fixed
86 void forceanglecalc(float _F) {
87
88     xVmulti = cos(pi*((90 - latangle) + XPLMGetDataf(phid)) / 180.0)*-1;
89     yVmulti = sin(pi*((90-latangle)+XPLMGetDataf(phid)) / 180.0)*-1;
90     yVmulti *= sin(pi*((90 - longangle) - XPLMGetDataf(thetaid)) / 180.0);
91     zVmulti = cos(pi*((90 - longangle) - XPLMGetDataf(thetaid)) / 180.0);
92     xVmulti *= sin(pi*((90 - longangle) - XPLMGetDataf(thetaid)) / 180.0);
93     Fx = _F * xVmulti;
94     Fy = _F * yVmulti;
95     Fz = _F * zVmulti;
96 }
97
98 //Calculating Forces and Moments from force vector applied at certian distance ...
    from CG
99 void forcecalc(float _Fx,float _Fy,float _Fz) {
100     fside = _Fx;
101     L = _Fx * Y - _Fy * X;
102     fnrml = _Fy;
103     M = _Fz * Y - _Fy * Z;
104     faxil = _Fz;
105     N = _Fz * X - _Fx * Z;
106 };
107
108 //Applies Force
109 void forceapply() {
110     XPLMSetDataf(fsidid, fside);
111     XPLMSetDataf(fnrmid, fnrml);
112     XPLMSetDataf(faxilid, faxil);
113     XPLMSetDataf(Lplgid, L);
114     XPLMSetDataf(Mplgid, M);
115     XPLMSetDataf(Nplgid, N);
116 };
117
118 PLUGIN_API int XPluginStart(
119     char *      outName,
120     char *      outSig,
121     char *      outDesc)
122 {
123     strcpy(outName, "ForcePlugin");
124     strcpy(outSig, "xpsdk.examples.helloworld3plugin");
125     strcpy(outDesc, "Plugin To simulate dynamic force on aircraft");
126
127     //HotKey
128     gHotKey = XPLMRegisterHotKey(XPLM_KEY_TAB, xplm_DownFlag,
129     "Applies Force",
130     MyHotKeyCallback,
131     NULL);
132
133     for (float t = 0; t < 4; t = t + 0.01) { //Finding time in Second
134         Order system where Force=Maximum Displacement Driven Force
135         F2 = 1.8607e+04 * (1.0 - (1.0 / (sqrt(1.0-(dr*dr))))*exp(-dr * w*t)*
136         sin(sqrt(1 -(dr*dr)) *w*t+ acos(dr)));
137         if (F2 ≥ 1.8607e+04) {
138             timedelay = t;
139             timedelay = timedelay - tforcechange; //Creating Time Delay for

```

```

140         force, so the the Second Order System Starts at the right force
141         break;
142     }
143 }
144 return 1;
145 }
146
147 PLUGIN_API void XPluginStop(void)
148 {
149
150     XPLMUnregisterHotKey(gHotKey);
151     XPLMDestroyFlightLoop(floopid);
152 }
153 }
154
155 PLUGIN_API void XPluginDisable(void) { }
156 PLUGIN_API int XPluginEnable(void) { return 1; }
157 PLUGIN_API void XPluginReceiveMessage(XPLMPluginID inFrom, int inMsg, void * ...
    inParam) { }
158
159
160 float MyFlightLoopCallback(
161     float          inElapsedSinceLastCall,
162     float          inElapsedTimeSinceLastFlightLoop,
163     int            inCounter,
164     void *         inRefcon)
165 {
166
167     if (count == 0) { //Starting Timer
168         time0=XPLMGetDataf(timeid);
169     };
170     time=XPLMGetDataf(timeid)-time0; //Time elapsed since start of force
171     if (time > 5) {
172         XPLMSpeakString("Finish");
173         countstr = std::to_string(XPLMGetDataf(timeid)); //Display at what time
174         force ended
175         char const *pchar = countstr.c_str();
176         XPLMSpeakString(pchar);
177         time = 0;
178         count = 0;
179         return 0;
180     }
181     if (time<tforcechange) { //Displacement driven
182         count = count + 1;
183         F1 = (2188.5*time*time)+(10212*time)+(143.67);
184         forceanglecalc(F1);
185         forcecalc(Fx, Fy, Fz);
186         forceapply();
187         return -1;
188     };
189     if (time ≥ tforcechange) { //Second Order System
190         F2= 1.8607e+04 * (1.0 - (1.0 / (sqrt(1.0 - (dr*dr))))*exp(-dr * ...
            w*(time+timedelay))
191             *sin((sqrt(1 - (dr*dr))*w*(time+timedelay))+ acos(dr)));
192         forceanglecalc(F2);
193         forcecalc(Fx, Fy, Fz);
194         forceapply();
195         return -1;
196     }

```

```
197     return -1;
198
199 }
200
201 void MyHotKeyCallback(void* inRefcon) {
202     XPLMCreateFlightLoop_t floop = { ...
203         sizeof(XPLMCreateFlightLoop_t), xplm.FlightLoop.Phase.AfterFlightModel,
204         MyFlightLoopCallback, nullptr };
205     floopid = XPLMCreateFlightLoop(&floop);
206     XPLMScheduleFlightLoop(floopid, 3, 1);
207 }
```